

Seconde Option SI	Mise en route — Aéroglisseur BLE ESP32-C3 + Web Bluetooth — Personnalisation des UUIDs	Travaux Pratiques
Durée : 15 min	Outils : GitHub, Thonny, Chrome ou Edge	Matériel : ESP32-C3, câble USB

Pourquoi modifier les UUIDs ?

Chaque binôme commande **son** aéroglisseur. Pour éviter les interférences entre binômes, chaque paire **ESP32** + **page HTML** doit avoir des identifiants BLE (UUIDs) uniques, générés par chaque groupe.

Étape 1 — Générer vos UUIDs

1. Aller sur uuidgenerator.net et générer **un seul UUID**.
Exemple obtenu : ddf00d17-b830-4252-af59-053af5305849
2. La **série** des 6 UUIDs se construit en incrémentant le **dernier chiffre du deuxième groupe** (ici b830) :

Variable firmware	UUID (exemple)	Votre UUID
SERVICE_UUID	ddf00d17-b830-4252-af59-053af5305849	
THROTTLE_UUID	ddf00d17-b831-4252-af59-053af5305849	
ARM_UUID	ddf00d17-b832-4252-af59-053af5305849	
STATUS_UUID	ddf00d17-b833-4252-af59-053af5305849	
SERVO_LR_UUID	ddf00d17-b834-4252-af59-053af5305849	
SERVO_FB_UUID	ddf00d17-b835-4252-af59-053af5305849	

Étape 2 — Forker le dépôt GitHub

1. Se connecter à **GitHub** avec votre compte.
2. Aller sur le dépôt du professeur (lien communiqué en classe).
3. Cliquer sur **Fork** (en haut à droite) → **Create fork**.
Vous obtenez votre propre copie du dépôt sous <https://votre-pseudo.github.io/>.

Étape 3 — Modifier les fichiers dans GitHub

Toutes les modifications se font **directement dans l'interface GitHub**, sans VS Code.

Modifier firmware/aeroglisseur_ble.py

Dans votre fork : ouvrir `firmware/aeroglisseur_ble.py` → cliquer sur l'icône **crayon** (Edit) → effectuer **deux modifications** → **Commit changes**.

1. Remplacer les 6 UUIDs par vos valeurs du tableau ci-dessus :

```
SERVICE_UUID = bluetooth.UUID("ddf00d17-b830-4252-af59-053af5305849")
THROTTLE_UUID = bluetooth.UUID("ddf00d17-b831-4252-af59-053af5305849")
ARM_UUID      = bluetooth.UUID("ddf00d17-b832-4252-af59-053af5305849")
STATUS_UUID   = bluetooth.UUID("ddf00d17-b833-4252-af59-053af5305849")
SERVO_LR_UUID = bluetooth.UUID("ddf00d17-b834-4252-af59-053af5305849")
SERVO_FB_UUID = bluetooth.UUID("ddf00d17-b835-4252-af59-053af5305849")
```

2. Changer le nom BLE (utilisé pour identifier votre ESP32 dans la liste) :

```
# Remplacer "Aeroglisseur" par votre prénom, ex :
name = b"Alice"
```

Modifier tp1_moteur.html (et tp2_direction.html)

Ouvrir le fichier → crayon → utiliser **Ctrl+H** (rechercher/remplacer dans le navigateur GitHub n'existe pas : repérer visuellement les UUIDs dans le bloc `const ... = "..."` en début de script) → remplacer chacun des 6 UUIDs → **Commit changes**.

Étape 4 — Charger le firmware sur l'ESP32 (Thonny)

1. Connecter l'ESP32-C3 au PC — ouvrir **Thonny**.
2. Vérifier : interpréteur = *MicroPython (ESP32)* (bas à droite).
3. Télécharger `aeroglisseur_ble.py` depuis votre fork GitHub.
4. Dans Thonny : **Fichier** → **Enregistrer sous...** → *Périphérique MicroPython* → nommer **main.py**.
5. Appuyer sur **Reset** de l'ESP32.
6. LED NeoPixel **rouge fixe** = prêt.

Pourquoi main.py ?

MicroPython exécute automatiquement `main.py` au démarrage.

Étape 5 — Branchement de l'ESC sur l'ESP32

L'ESC communique avec l'ESP32 par un **signal PWM 50 Hz** sur un seul fil. Le connecteur de l'ESC comporte 3 fils à brancher sur l'ESP32 :

Fil ESC	Broche ESP32-C3	Rôle
 noir (-)	GND	Masse commune — obligatoire même si l'ESC est alimenté séparément
 rouge (+)	<i>Ne pas connecter</i>	Alimentation 5 V BEC — l'ESP32 est alimenté par USB, ne pas relier
 jaune/blanc (signal)	GPIO 4	Signal PWM 50 Hz (0,5 ms à 2,5 ms)

Attention — ne pas connecter le fil rouge

Le fil rouge du connecteur ESC fournit une tension de 5V (BEC interne). Si l'ESP32 est simultanément alimenté par USB, connecter ce fil rouge peut endommager le régulateur de tension de l'ESP32 ou du PC. **Laisser le fil rouge non connecté.**

Rappel des broches utilisées (firmware aeroglisseur_ble.py)

GPIO 4 Signal ESC (PWM 50 Hz)
GPIO 5 Servo gauche/droite
GPIO 6 Servo avant/arrière
GPIO 8 LED NeoPixel (témoin d'état)

Étape 6 — Vérifier la connexion BLE

1. Ouvrir https://votre-pseudo.github.io/.../tp1_moteur.html dans **Chrome** ou **Edge**.
2. Cliquer **Connecter l'ESP32** → sélectionner *Aeroglisseur*.
3. LED bleu pâle = connexion réussie.

Web Bluetooth ne fonctionne pas ?

- Navigateur = Chrome ou Edge (Firefox non supporté).
- URL en **HTTPS** (pas `file://`).
- Les 6 UUIDs du HTML = les 6 UUIDs du firmware (vérifier la saisie).

Question — États de démarrage de l'ESP32

Q — Phases de démarrage et couleurs de la LED

En vous aidant de `aide_memoire_esc.html`, compléter le tableau des états successifs de la LED NeoPixel lors de la procédure d'armement :

État	Description	Couleur LED	Action à réaliser
0	En attente de connexion BLE	 rouge	Ouvrir la page web
1			
2			
3			
4			

Ressources

Fichier / Page	Contenu
<code>tp1_moteur.html</code>	Interface de commande : armement ESC 5 étapes, curseur de gaz, joystick NippleJS.
<code>aide_memoire_esc.html</code>	États LED, procédure de calibration PWM, paramètres progCard, specs XC-12-Lite.
<code>firmware/aeroglisser_ble.py</code>	Firmware MicroPython à charger sur l'ESP32 (enregistrer sous <code>main.py</code>).

Checklist avant de commencer le TP

- ☐ UUID généré sur `uuidgenerator.net` — tableau complété.
- ☐ Fork du dépôt réalisé sur GitHub.
- ☐ `aeroglisser_ble.py` modifié (6 UUIDs) et commité.
- ☐ `tp1_moteur.html` modifié (6 UUIDs) et commité.
- ☐ Firmware chargé sur l'ESP32 sous le nom `main.py` — LED rouge.
- ☐ Connexion BLE vérifiée depuis GitHub Pages — LED bleue.
- ☐ `aide_memoire_esc.html` ouvert dans un second onglet.